

Reflective

A programming language I created
(Compiler and interpreter deliverables)

The name currently comes from the idea of features being conceptually simple, but can be implemented to a high level of abstraction, allowing many different use cases. If you can think of a better name, let me know

Introduction

Making a programming language is difficult!

Many topics to explore:

- How computers work at a lower level
- What might make a programming language a viable for a software engineer?
- How well can an AI agent do on my language?

-Generative AI was used to answer my endless questions about the concepts relating to compilers, and AI agents were used to write code in my language (elaborated on later) as part of stress testing.

-About 90% of C++ code written was written the good old way, as my goal was to deepen my understanding of these concepts.

-In fact, after networking with engineers, I have been told AI agents still struggle a lot on C++ opposed to other languages (even in late 2026). This gave me some inspiration for the last bullet point.

Syntax Overview

Notably, functions can be stored in arrays, passed as arguments to functions, and returned from functions.

```
1 fn double(a : Int) -> Int { return a * 2; }
2 fn square(a : Int) -> Int { return a * a; }
3 fn fib(a : Int) -> Int {
4   if(a <= 2){ return 1; }
5
6   return fib(a-1) + fib(a-2);
7 }
8
9 let cool_functions : DynamicArray<fn(Int) -> Int> = [double, square, fib];
10
11 fn map(
12   function : fn(Int) -> Int,
13   data : DynamicArray<Int>
14 ) -> DynamicArray<Int> {
15   let result : DynamicArray<Int> = [];
16
17   for(element : Int in data){
18     push(result, function(element));
19   }
20   return result;
21 }
22
23 let cool_array : DynamicArray<Int> = [10, 20, 30];
24
25 for(function : fn(Int) -> Int in cool_functions){
26   print(map(function, cool_array));
27 }
28
```

Output
20 40 60
100 400 900
55 6765 832040

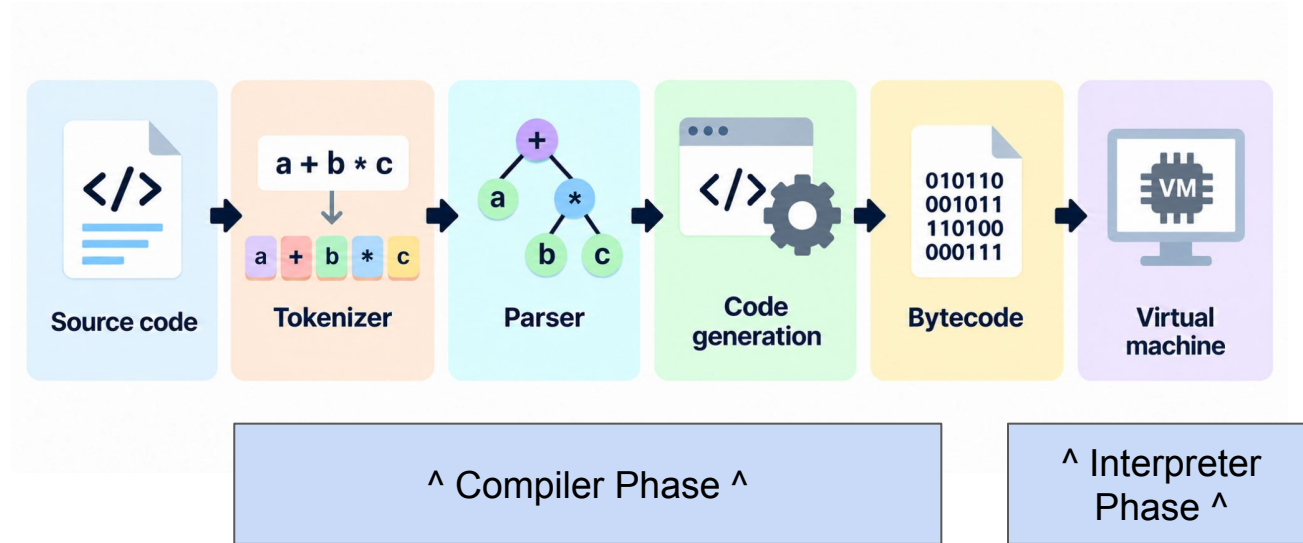
Figure 1: Code snippet of first class function behaviour

Execution model

Execution model
similar to Java

Execution split into
modules (Compiler &
VM) allows simplicity,
as well as moderately
strong performance

Implemented in C++.



Tokenizer and Parser

Tokenizer transforms text file containing source code into a format more friendly to the parser.

Parser reads every statement and expression recursively, and generates an abstract syntax tree (AST).

```
fn positiveMod(value : Int, modulus : Int) -> Int {  
  let result : Int = value - (value / modulus) * modulus;  
  if(result < 0){ result = result + modulus; }  
  return result;  
}
```

Figure 1: Function being analysed

```
--Function positiveMod -> int  
|--Parameter value: int  
|--Parameter modulus: int  
|--Block  
|--Let result: int  
|--Binary -  
|--Variable value  
|--Binary *  
|--Binary /  
|--Variable value  
|--Variable modulus  
|--Variable modulus  
|--If  
|--Binary <  
|--Variable result  
|--Integer 0  
|--Block  
|--Assignment result  
|--Binary +  
|--Variable result  
|--Variable modulus  
|--Return  
|--Variable result
```

Figure 2: Parser output when printed to terminal

Compiling/Code Generation

- The compiler traverses the output of the parser
- Every node is compiled recursively

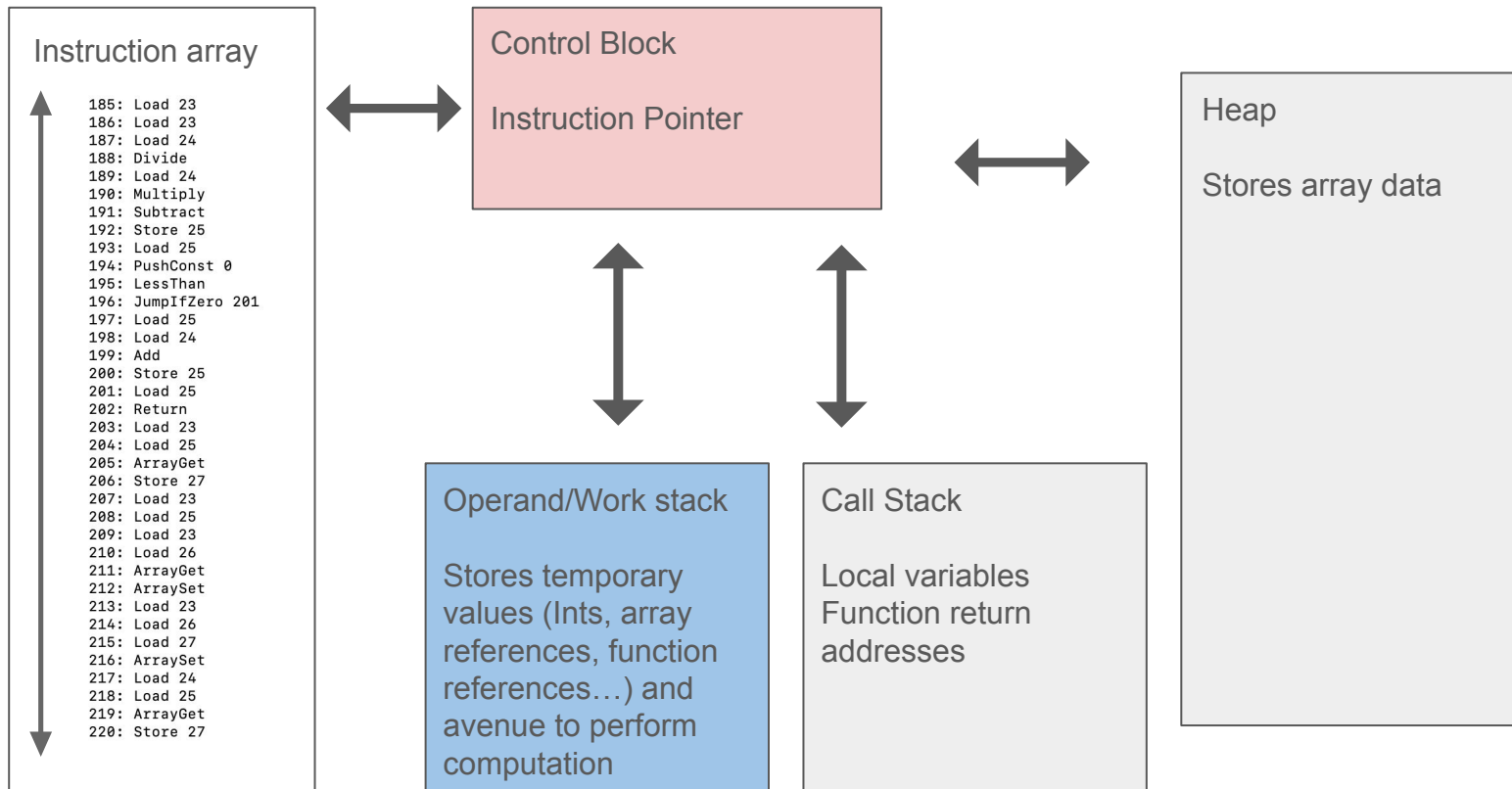
```
fn positiveMod(value : Int, modulus : Int) -> Int {  
  let result : Int = value - (value / modulus) * modulus;  
  if(result < 0){ result = result + modulus; }  
  return result;  
}
```

```
if (const auto* ifStmt = dynamic_cast<const IfStmt*>(&statement)) {  
  compileExpr(*ifStmt->conditional);  
  
  const std::size_t jump_index = code.code.size();  
  emit(OpCode::JumpIfZero); Jump past the body of the if statement if and only if  
                           the conditional expression evaluates to zero (false).  
  
  compileBlockStatement(*ifStmt->body_);  
  code.code[jump_index].a = static_cast<std::int32_t>(code.code.size());  
  return;  
}
```

**Compiles the expression for the conditional, as well
as the body of the if statement.**

Figure 1: Compiling an if statement

Virtual Machine

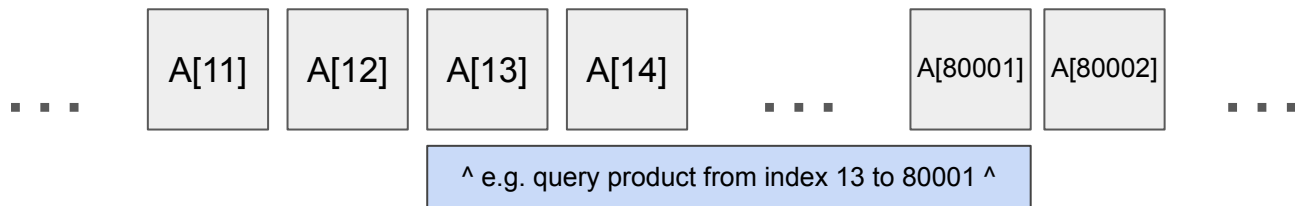


Can an AI agent understand it?

(Deepseek v4.1 Flash via OpenCode, Low reasoning)

Prompt: Implement a program that contains an array of 2x2 matrices $A[1]$, $A[2]$, $A[3]$, ..., $A[100,000]$

The program can answer online queries that either update the matrix at index i , or query the matrix product $A[i] \times A[i+1] \times \dots \times A[j]$ (query input is the start and stop indices i and j). Run the program on 100,000 random queries. Approximately 80% of queries should be updates.



To avoid overflow, all results are computed modulo $1e9+7$

Can an AI agent understand it?

(Deepseek v4.1 Flash via OpenCode, Low reasoning)

Prompt: Implement a program that contains an array of 2x2 matrices $A[1], A[2], A[3], \dots, A[100,000]$

The program can answer online queries that either update the matrix at index i , or query the matrix product $A[i] \times A[i+1] \times \dots \times A[j]$ (query input is the start and stop indices i and j). Run the program on 100,000 random queries. Approximately 80% of queries should be updates.

```
Function 7 (update)
  index -> 19
  a -> 20
  b -> 21
  c -> 22
  d -> 23
  matrix -> 24
  position -> 25
Function 8 (hashRangeProduct)
  start -> 19
  stop -> 20
  previousHash -> 21
  first -> 22
  last -> 23
  result -> 24
Output
31194624
VM execution time: 21051.6 ms
```

Under no input/output, and randomised initialization and queries, the program took 21 seconds. (0.21ms per query).

To avoid overflow, all results are computed modulo $1e9+7$

Stress Testing

(Continuing from previous slide)

Modify the program, so every matrix is initialised to $[1 \ 1 \ 1 \ 0]$. Then perform 40 queries that will give the first 40 fibonacci numbers. Then perform 10 more queries for the 10k, 20k, ... 100 kth fibonacci numbers.

Output

1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765
10946
17711
28657
46368
75025
121393
196418
317811
514229
832040
1346269
2178309
3524578

425147726
707832415
599151797
86092474
911435502

VM execution time: 1353.25 ms
lukebartram@Lukes-MacBook-Pro-4 lang %

- First 40 fibonacci numbers are small enough to be verified easily.
- The program computes all results modulo $1e9+7$ to avoid overflow issues. The 100,000th fibonacci number was verified independently via C++.

Design features and next steps

AI agents are smart, but only to begin with! How they differ from humans:

- Conversation quality degrades sharply as context window size increases
- **Reducing the need for reasoning tokens** also reduces context rot (and is cheaper and faster too)

Therefore,

- Keep language features orthogonal, predictable, consistent
- Abstractions allow a higher “signal to noise” ratio of source code

Concrete features to do: Generic types/functions/metaprogramming, reflection capability. Formal verification.

Conclusion

- Had a lot of fun working on this project and am looking forward to implementing more features.
- Satisfying to observe the possibilities that emerge from the addition of simple features
- Any questions?